Celo

Security Review

Cantina Managed review by:

0xRajeev, Lead Security Researcher
Cryptara, Security Researcher

July 1, 2025

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**

3 Findings **4**
3.1 Informational . . . 4
3.1.1 Redundant L1 logic in `PrecompilesOverride` reduces readability . . . 4
3.1.2 Obsolete function usage in `IMockValidators` affects validation . . . 4
3.1.3 Unused `HotfixWhitelisted` event in `Governance` contract can be removed . . . 4

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	*Must* fix as soon as possible (if already deployed).
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Celo is an Ethereum L2, scaling real-world solutions & leading a thriving new digital economy for all.

From May 24th to May 27th the Cantina team conducted a review of celo-monorepo. The review focused on the differences between tags `core-contracts.v12` (current latest audited and released version) and `ore-contracts.v13.pre-audit` (in essence, a `diff` between commit hash 0374f281 and 807c5cc7).

The particular scope included the following files (and their dependencies):

- `contracts-0.8/common/CeloUnreleasedTreasury.sol`.
- `contracts-0.8/common/EpochManager.sol`.
- `contracts-0.8/common/SuperBridgeETHWrapper.sol`.
- `contracts-0.8/common/interfaces/IStandardBridge.sol`.
- `contracts-0.8/common/interfaces/IWETH.sol`.
- `contracts-0.8/common/libraries/ReentrancyGuard08.sol`.
- `contracts-0.8/governance/Validators.sol`.
- `contracts-0.8/governance/interfaces/IValidatorsInitializer.sol`.
- `contracts/common/Accounts.sol`.
- `contracts/common/GoldToken.sol`.
- `contracts/common/Permissioned.sol`.
- `contracts/common/interfaces/IAccounts.sol`.
- `contracts/common/interfaces/ICeloToken.sol`.
- `contracts/governance/Election.sol`.
- `contracts/governance/EpochRewards.sol`.
- `contracts/governance/Governance.sol`.
- `contracts/governance/GovernanceSlasher.sol`.
- `contracts/governance/ReleaseGold.sol`.
- `contracts/governance/interfaces/IElection.sol`.
- `contracts/governance/interfaces/IEpochRewards.sol`.
- `contracts/governance/interfaces/ILockedCelo.sol`.
- `contracts/governance/interfaces/IReleaseGold.sol`.
- `contracts/governance/interfaces/IValidators.sol`.

The team identified a total of **3** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	1
Informational	3	2	0
Total	**3**	**2**	**1**

3 Findings

3.1 Informational

3.1.1 Redundant L1 logic in `PrecompilesOverride` reduces readability

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `Election` contract inherits from `PrecompilesOverride`, which defines a function `numberValidatorsInCurrentSet` with branching logic based on the `isL2()` check. However, the codebase has been migrated to operate exclusively in an L2 context. Despite this architectural shift, the function retains the `if (isL2()) { ... } else { ... }` conditional and a fallback to `super.numberValidatorsInCurrentSet()`. Given that L1 compatibility is no longer required, this bifurcation introduces unnecessary complexity and misleading assumptions about the deployment environment. Any invocation of the fallback L1 logic is not only obsolete but also incurs a readability and maintainability cost.

This pattern is not isolated; several functions in `PrecompilesOverride` (e.g., `getEpochNumberOfBlock`, `validatorSignerAddressFromCurrentSet`) follow similar dual-path logic.

Recommendation: The function `PrecompilesOverride.numberValidatorsInCurrentSet` should be refactored to remove the `isL2()` check and fallback to `super`. Instead, it should directly return the result of the canonical L2 source:

```
function numberValidatorsInCurrentSet() public view returns (uint256) {
  return getEpochManager().numberOfElectedInCurrentSet();
}
```

This change aligns the implementation with the stated L2-only design goal and reduces cognitive overhead for future maintainers. If similar L1/L2 logic persists in other overridden functions that are now guaranteed to run only on L2, those should be considered for simplification as well.

Celo: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 Obsolete function usage in `IMockValidators` affects validation

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `calculateGroupEpochScore()` still exists in the `IMockValidators` interface and is used in the test suite (`governance_tests.ts`), but was removed from the `Validators.sol` contract. This means the test logic is relying on a function that no longer exists in production, making it outdated, potentially misleading and likely resulting in test failure.

Recommendation: `calculateGroupEpochScore()` should be removed from the mock interface and any related test logic should be updated to reflect the current implementation. If scoring is still needed in tests, it should be computed manually or replaced with supported contract methods.

Celo: Fixed in PR 11428.

Cantina Managed: Reviewed that PR 11428 removes `calculateGroupEpochScore()` from the `IMockValidators` interface.

3.1.3 Unused `HotfixWhitelisted` event in `Governance` contract can be removed

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The `Governance` contract defines an event `HotfixWhitelisted`, intended for use in the legacy L1 governance model where hotfixes required individual whitelist approvals from validator signers. This event is tied to an obsolete part of the `HotfixRecord` structure (`deprecated_whitelisted`) and is never emitted by any function in the current contract.

Recommendation: Remove the `HotfixWhitelisted` event from the `Governance` contract, along with any related deprecated state (`HotfixRecord.deprecated_whitelisted`, `deprecated_preparedEpoch`) unless it is being used for backward compatibility or migration-specific read access/log parsing.

Celo: Fixed in PR 11427.

Cantina Managed: Reviewed that PR 11427 removes `HotfixWhitelisted` event.